TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC

BÁO CÁO BÀI TẬP TUẦN 1

MÔN: THỊ GIÁC MÁY TÍNH (MAT3562E 2)

LAB 01: XỬ LÝ ẢNH CƠ BẢN

HISTOGRAM VÀ POINT PROCESSING

Giảng viên: TS. Cao Văn Chung
Sinh viên: Bùi Quang Chiến
MSSV: 23001837
Lớp: MAT3562E 2

Hà Nội, tháng 2 năm 2026

Mục lục

1 Giới thiệu **2**
1.1 Mục tiêu 2

2 Lý thuyết cơ bản **2**
2.1 Histogram 2
2.2 Point Processing 2

3 Bài tập về nhà **2**
3.1 Problem 1: Gray Level Modification 2
3.2 Problem 2: Median Thresholding 3
3.3 Problem 3: Power Transform 3

4 Kết quả thực nghiệm **4**
4.1 Ảnh gốc và histogram 4

5 So sánh và kết luận **5**
5.1 So sánh các phương pháp 5
5.2 Kết luận 5

6 Tài liệu tham khảo **5**

1 Giới thiệu

Bài lab này tập trung vào các kỹ thuật xử lý ảnh cơ bản trong Computer Vision, bao gồm:

- Phân tích histogram và phân bố cường độ pixel
- Các phép biến đổi điểm ảnh (Point Processing)
- Kỹ thuật tăng cường chất lượng ảnh

1.1 Mục tiêu

1. Hiểu về cấu trúc ma trận ảnh và histogram
2. Thực hiện các phép biến đổi: Negative, Log Transform, Power Transform
3. Áp dụng kỹ thuật Thresholding để phân đoạn ảnh
4. So sánh hiệu quả của các phương pháp

2 Lý thuyết cơ bản

2.1 Histogram

Histogram là biểu đồ thể hiện phân bố cường độ pixel trong ảnh:

- **Trục X:** Giá trị cường độ (0-255)
- **Trục Y:** Số lượng pixel có giá trị đó

2.2 Point Processing

Point Processing là phương pháp biến đổi từng pixel độc lập theo công thức:

$$s = T(r)$$

trong đó r là giá trị pixel đầu vào, s là giá trị pixel đầu ra.

3 Bài tập về nhà

3.1 Problem 1: Gray Level Modification

Công thức:

$$s = 16 \times \sqrt{r}$$

Code Python:

```python
def GrayLevelModification(img_input):
    f = img_input.astype(np.float32) / 255.0
    s = 16.0 * np.sqrt(f)
    s_normalized = s / s.max()
    return (255.0 * s_normalized).clip(0, 255).astype(np.uint8)
```

Giải thích: Hàm căn bậc hai làm sáng ảnh từ từ, hệ số 16 đảm bảo output nằm trong khoảng [0, 255].

3.2 Problem 2: Median Thresholding

Định nghĩa: Median m thỏa mãn $P(x < m) = P(x > m)$

Code Python:

```
def MedianThreshold(img_input):
    hist = cv2.calcHist([img_input], [0], None, [256], [0, 256]).ravel()
    cum_hist = np.cumsum(hist)
    total_pixels = cum_hist[-1]
    median_idx = np.searchsorted(cum_hist, total_pixels / 2.0)
    img_thresh = np.where(img_input >= median_idx, 255, 0).astype(np.uint8)
    return img_thresh, median_idx
```

Giải thích: Tính cumulative histogram và tìm vị trí mà tổng tích lũy bằng 50% tổng số pixel.

3.3 Problem 3: Power Transform

Công thức:

$$s = c \times r^{\gamma}$$

Code Python:

```
def PowerTransform(img_input, gamma, c=1.0):
    f = img_input.astype(np.float32) / 255.0
    s = c * np.power(f, gamma)
    return (255.0 * s).clip(0, 255).astype(np.uint8)
```

Ảnh hưởng của γ:

- γ bé hơn 1: Làm **Sáng** ảnh hơn.
- γ lớn hơn 1: Làm **Tối** ảnh đi.
- γ bằng 1: Không thay đổi chất lượng hay độ sáng của ảnh.

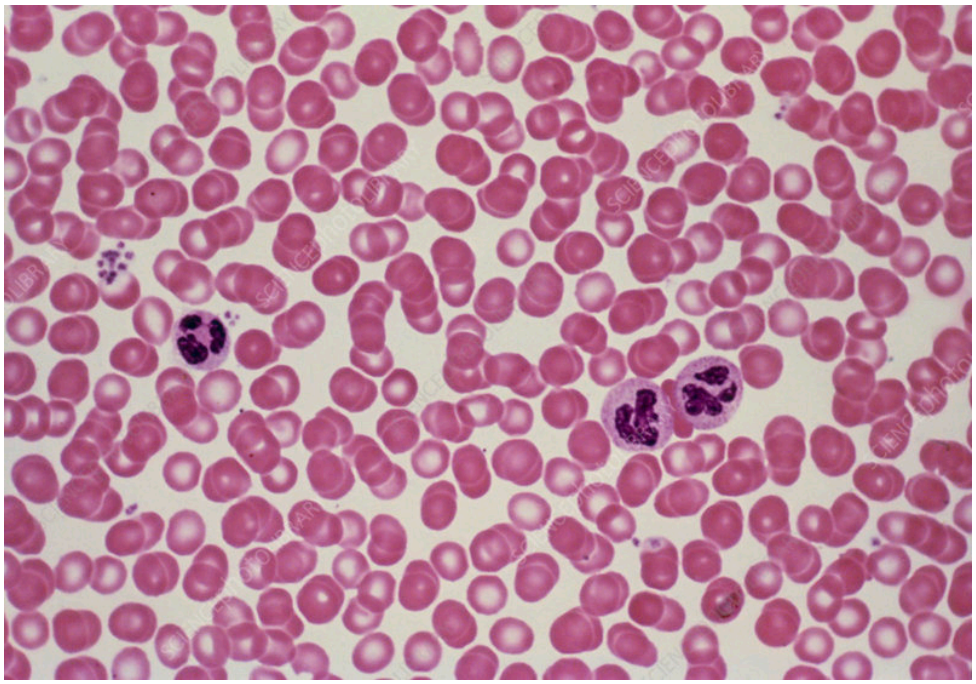
Kết quả thực nghiệm:

- **runway.jpg:** $\gamma = 0.7$ cho kết quả tốt nhất
- **spine.jpg:** $\gamma = 0.4 - 0.7$ phù hợp cho ảnh y tế

4 Kết quả thực nghiệm

4.1 Ảnh gốc và histogram

Hình 1: Kết quả các phép biến đổi trên ảnh runway.jpg



Hình 2: Kết quả các phép biến đổi trên ảnh spine.jpg

5 So sánh và kết luận

5.1 So sánh các phương pháp

Bảng 1: So sánh các phương pháp xử lý ảnh

Phương pháp	**Ưu điểm**	**Nhược điểm**
Gray Level Mod	Đơn giản, luôn làm sáng	Không linh hoạt
Median Threshold	Phân đoạn nhanh	Mất thông tin
Power Transform	**Linh hoạt nhất**	Cần chọn γ

5.2 Kết luận

- **Power Transform là phương pháp tốt nhất** vì có thể điều chỉnh γ phù hợp với từng loại ảnh
- Ảnh y tế (spine): Dùng $\gamma = 0.4 - 0.7$
- Ảnh tự nhiên (runway): Dùng $\gamma = 0.7$
- Histogram giúp phân tích phân bố cường độ pixel hiệu quả

6 Tài liệu tham khảo

1. OpenCV Documentation: https://docs.opencv.org/
2. NumPy Documentation: https://numpy.org/doc/